🏥 Healthcare Management System

(Microservices Architecture – ASP.NET Core + Angular)

📌 1. Project Objective

Build a **Healthcare Management System** using **Microservices Architecture** where:

- Users can log in with roles (Admin / Doctor / Patient)
- Patients can book appointments
- Doctors can manage and complete appointments
- System tracks **appointment lifecycle using Status**

👉 Focus: Learn **Microservices + API Gateway + Role-Based Access**

🧱 2. Technology Stack

Backend

- ASP.NET Core Web API
- Entity Framework Core (Code First)
- SQL Server
- JWT Authentication

Frontend

- Angular

Architecture

- Microservices
- API Gateway (Ocelot recommended)

📐 3. System Architecture (Simple View)

🔄 4. Request Flow

1. User logs in → receives JWT token
2. Angular sends request → API Gateway
3. Gateway routes → correct microservice
4. Service processes request → database
5. Response → UI

📦 5. Microservices Design

Each service follows:

Controller → Service → Repository → Database

👤 6. User Service (Authentication + Roles)

🎯 Purpose

- Register & Login users
- Assign roles

👥 Roles

- Admin
- Doctor

- Patient

📜 **Entity**

```
public class User
{
    public int Id { get; set; }
    public string Username { get; set; }
    public string Password { get; set; }
    public string Role { get; set; }
}
```

👱 **7. Patient Service**

🎯 **Purpose**

Manage patient details

📜 **Entity**

```
public class Patient
{
    public int Id { get; set; }
    public string Name { get; set; }
    public int Age { get; set; }
}
```

👨‍⚕️ 8. Doctor Service

🎯 Purpose

Manage doctor details

📜 Entity

```
public class Doctor
{
    public int Id { get; set; }
    public string Name { get; set; }
    public string Specialization { get; set; }
}
```

🗓️ 9. Appointment Service (Core Module)

🎯 Purpose

Handle booking and lifecycle of appointments

📜 Appointment Status (IMPORTANT)

```
public enum AppointmentStatus
{
    Booked = 1,
    Cancelled = 2,
    Completed = 3
}
```

🗒️ Appointment Entity

```C#
public class Appointment
{
    public int Id { get; set; }

    public int PatientId { get; set; }
    public int DoctorId { get; set; }

    public DateTime AppointmentDate { get; set; }

    public AppointmentStatus Status { get; set; }
}
```

👉 Default Status = Booked

🔄 Appointment Lifecycle

Booked → Completed (Doctor)

→ Cancelled (Patient / Doctor)

🔐 Role-Based Actions

Role	Actions
Patient	Book, Cancel
Doctor	View, Complete, Cancel
Admin	Full Control

⚙️ Business Rules

- Patient can only **cancel**
- Doctor can **complete or cancel**

- Cannot:
    - Cancel completed appointment
    - Complete cancelled appointment

🚪 10. API Gateway

🎯 Purpose

Single entry point for all services
Configure Authentication Token

🗄️ 11. Database Strategy

Each service has its own DB:

- UserDB
- PatientDB
- DoctorDB
- AppointmentDB

👉 Ensures loose coupling

🔗 12. Service Communication

Simple approach:

- REST API calls
- Example:
    - Appointment Service → validates DoctorId via Doctor Service

---

📜 User Stories – Healthcare System (Simplified)

👤 User Service

ID	User Story	Role	Acceptance Criteria
US1	Register user	All	Username unique, role assigned (Admin/Doctor/Patient)
US2	Login user	All	Valid credentials return JWT token
US3	Role-based access	System	APIs restricted based on role

👱 Patient Service

ID	User Story	Role	Acceptance Criteria
PS1	Create profile	Patient	Patient details saved
PS2	View profile	Patient/Admin	Patient sees own, Admin sees all
PS3	Update profile	Patient	Only own data updated

👨‍⚕️ Doctor Service

ID	User Story	Role	Acceptance Criteria
DS1	Add doctor	Admin	Doctor created with specialization
DS2	View doctors	All	List of doctors available
DS3	Update doctor	Admin	Doctor details updated

📅 Appointment Service

ID	User Story	Role	Acceptance Criteria
AS1	Book appointment	Patient	Created with Status = Booked
AS2	Cancel appointment	Patient	Only own, Booked → Cancelled
AS3	Complete appointment	Doctor	Booked → Completed
AS4	Cancel appointment	Doctor	Status → Cancelled
AS5	View appointments	All	Patient (own), Doctor (assigned), Admin (all)
AS6	Status validation	System	No invalid transitions (Completed/Cancelled rules)

🌐 API Gateway

ID	User Story	Role	Acceptance Criteria
GW1	Route requests	System	Requests go to correct service
GW2	Validate JWT	Gateway	Invalid token blocked

🎨 13. Angular Frontend

Modules:

- Login
- Patient
- Doctor
- Appointment

UI Actions:

- Patient:

    - Book appointment
    - Cancel appointment
- Doctor:
    - View appointments
    - Mark as completed

📁 14. Project Structure

```
/HealthcareSolution
   /Gateway
   /UserService
   /PatientService
   /DoctorService
   /AppointmentService
   /AngularApp
```

🧪 15. Testing

- Swagger
- Postman

🚀 16. Student Implementation Tasks

1. Create 4 Microservices
2. Implement EF Core (Code First)
3. Add JWT Authentication
4. Configure API Gateway
5. Connect Angular UI
